

Measuring Investment Risks

In this chapter, we will be discussing how to measure investment risks. When we think about an investment, we have to consider both its potential advantages and its possible downfalls. In other words, we should bear in mind both the profit that will be made if everything goes well, and the potential losses that might occur if the investment is unsuccessful.

In this chapter, we will use a very structured approach. We will explain concepts from a theoretical perspective first, in order to understand their meaning and purpose, and then we'll look at some actual calculations using Python. In the first few sections of this chapter, we will focus on the rates of return of an investment. We will start by learning how to calculate the rate of return of a single security, then we'll continue by calculating the rate of return of a portfolio of securities. After that, we'll explain stock indices and how they are composed. We'll also learn about risk and the statistical measures that allow us to calculate the standard deviation and variance of risk. Following this, we'll examine the relationship between two securities, also known as correlation, and we will learn how to calculate the correlation and covariance of two securities.

This chapter focuses on the following topics:

- Calculating the return of a single security
- Calculating the return of a portfolio of securities
- Risk, standard deviation, and variance
- Correlation and covariance

Technical requirements

In this chapter, we will be using the Jupyter Notebook for coding Purposes. We Will also be using the pandas and Numpy libraries.

Before moving on, let's take a look at the different sources from which we can extract financial data.

Sources of financial data

Let's look at how to use financial data from a technical perspective. The data you'll be using for financial analysis, or for any other type of analysis, is likely to come from one of two sources: a web server or your computer. In practice, in order to access data stored on a web server, you'll need to connect to its **application programming interface (API)**. In our case, we will need a financial data API. We can also call these online financial data sources; examples of these include the IEX, Morningstar, Alpha Vantage, and the Quandl API.

An important characteristic of these servers is that they provide up-to-date data. For this reason, you'll need an internet connection to access these APIs. The pandas datareader is an example of a module that will help you retrieve data from financial data sources and prepare it for analysis. Apart from extracting online data, you may use information that is stored as certain types of files on your computer. One file format that every programmer or analyst needs to know how to work with is the **comma-separated values (CSV)** file. We used this type of file in the previous chapter for our stock forecasting using the ARIMA model. It is important to know that neither of these two approaches are straightforward. In the next few sections, we will clarify how these two methods work and we will introduce you to the relevant programming tools. This will be a valuable addition to your skill set.

It is usually much harder to clean and organize your data than to conduct the subsequent business or financial analysis. At first glance, it might seem like a better idea to use APIs as these provide the better

data, and all you need is an internet connection. So, why bother using CSV files at all? Firstly, web services are prone to breaking down for unknown periods of time. Secondly, it is possible that a certain API may contain only part of the data that you need for proper financial analysis. Currently, there's no free API out there that provides financial data that is as rich as you might want. Some APIs offer single or multiple stock data, while others provide foreign stock data or data to do with market indices. Note that an analyst may often need to work with all of these. Furthermore, you can sometimes only connect to an API if you use Python 3, and not Python 2.

Taking these drawbacks into consideration, most of the time, we will be using various CSV files that are present in the GitHub repository of this chapter. Although the information contained in the CSV file may not be up to date, it will be the most appropriate as it is the richest and easiest-to-handle data that we can provide. This will allow you to focus on coding, financial theory, and the relation between the two, as we have already mentioned. If you insist on working with up-to-date financial data, however, it is suggested that you use the Morningstar or Alpha Vantage API if you are using Python 2 or Python 3. You should only use IEX if you are on Python 3. Let's now go ahead and look at some examples of how we can extract or retrieve financial data from various sources.

In order to read the financial data from the API, we will be using the pandas-datareader library. To use this, we need to install this library; type the following command in the Anaconda Command Prompt:

```
conda install pandas-datareader
```

Once you install the package, we will be reading from various financial APIs, such as IEX and Morningstar. First, we will import the pandas and pandas_datareader libraries. While using this API, we will retrieve

details related to Apple, which is denoted as AAPL in the stock index:

```
In [5]:
import pandas as pd
pd.core.common.is_list_like = pd.api.types.is_list_like
from pandas_datareader import data as wb
```

Make sure you execute the second line, otherwise you may get an error. This line indicates that we should *bypass* the version control of your package manager. We will be using the DataReader function to read from the financial API. The basic syntax of this function is given as follows:

```
In [28]: GOOGL = wb.DataReader('GOOGL', data_source='morningstar', start='2015-1-1')
In [29]: Signature: wb.DataReader(name, data_source=None, start=None, end=None, retry_count=3, pau
Out[29]: se=0.001, session=None, access_key=None)
Docstring:
Imports data from a number of online sources.
```

The following are the details of some of the specified parameters:

Parameters	Description
name	This is the name of the dataset. Some data sources, such as Google or FRED, will accept a list of names.
datasource	This is the name of the financial API, such as Morningstar or IEX.
start	This parameter specifies where we want to start reading the financial data from.
end	This parameter specifies where we should stop reading the financial data.

Let's now go ahead and see whether we can retrieve the data from the financial API. The code to do this is as follows:

```
In [9]:
AAPL_IEX = wb.DataReader('AAPL', data_source='iex', start='2015-1-1')

Out[9]:
5y
```

The following code helps us to see the top five records of the AAPL_IEX dataframe:

```
In [10]:
AAPL_IEX.head()
```

The output is as follows:

Out[10]:

	open	high	low	close	volume
date					
2015-01-02	104.2471	104.2939	100.4662	102.3192	53204626
2015-01-05	101.3459	101.6828	98.6506	99.4367	64285491
2015-01-06	99.7082	100.5411	97.9206	99.4461	65797116
2015-01-07	100.3258	101.2617	99.8532	100.8406	40105934
2015-01-08	102.2257	104.9584	101.7296	104.7151	59364547

In the preceding code, our goal was to extract data from the IEX data source about AAPL from January 1, 2015. The DataReader function allows us to do this in one row. The wb alias uses this DataReader function and we specify three important parameters. The first one is the ticker of the Apple stock, AAPL. Second, we select the data source, Morningstar. Finally, we specify the start date, which is January 1, 2015.

With respect to these parameters, we get the preceding output.

We can use the same function to retrieve details about the stock price of Google using the GOOGL ticker, as shown in the following code block:

```
In [12]:
GOOGL_IEX = wb.DataReader('GOOGL', data_source='iex', start='2015-1-1')

Out[12]:
5y
```

The following code helps us to see the top five records:

```
In [13]:
GOOGL_IEX.head()
```

The output is as follows:

Out[13]:

	open	high	low	close	volume
date					
2015-01-02	532.60	535.8000	527.88	529.55	1327870
2015-01-05	527.15	527.9899	517.75	519.46	2059119
2015-01-06	520.50	521.2100	505.55	506.64	2731813
2015-01-07	510.95	511.4900	503.65	505.15	2345875
2015-01-08	501.51	507.5000	495.02	506.91	3662224

The output that we see indicates that we can retrieve a maximum of the past five years of data.

We can also use different data sources, such as Morningstar, to retrieve financial data, as shown here:

```
In [20]:  
AAPL = wb.DataReader('AAPL', data_source='morningstar', start='2015-1-1')
```

The following code helps us to see the top five records:

```
In [22]:  
AAPL.head()
```

The output is as follows:

Out[22]:

		Close	High	Low	Open	Volume
Symbol	Date					
AAPL	2015-01-01	110.38	110.38	110.380	110.38	0
	2015-01-02	109.33	111.44	107.350	111.39	53204626
	2015-01-05	106.25	108.65	105.410	108.29	64285490
	2015-01-06	106.26	107.43	104.630	106.54	65797116
	2015-01-07	107.75	108.20	106.695	107.20	40105934

The following code helps us to read the dataset from the

Morningstar data source:

```
In [28]:  
GOOGL = wb.DataReader('GOOGL', data_source='morningstar', start='2015-1-1')
```

The following code helps us to see the top five records:

```
In [29]:  
GOOGL.head()
```

The output is as follows:

Out[29]:

		Close	High	Low	Open	Volume
Symbol	Date					
GOOGL	2015-01-01	530.7	530.7	530.7	530.7	0
	2015-01-02	529.6	535.8	527.9	532.6	1327870
	2015-01-05	519.5	528.0	517.8	527.2	2059119
	2015-01-06	506.6	521.2	505.6	520.5	2731813
	2015-01-07	505.2	511.5	503.7	511.0	2345875

Security risks and returns

One of the most important questions asked in this domain is, why do different investments have different levels of risk and profitability? The answer to this question can be easily understood if we compare stocks and bonds. Let's consider an example: government bonds conventionally offer an average rate of return of 3%. Historically, there have been very few cases of governments going bankrupt and not recompensing what's owed to investors. If an investor buys a government debt, they can be certain that one year from now they will obtain their initial investment plus the expected interest. While this investment isn't risk-free, the risk is contained. Equity shares have a higher rate of return—approximately 6%. However, they are associated with much more frequent fluctuations and price changes, as different variables influence a company's share price.

The art of finance isn't about maximizing an investor's returns in a year. It is about making carefully considered decisions that take into account both risk and return, and optimizing an investment portfolio. Now that we have understood the importance of risk and return, we are ready to learn how to calculate the two parameters.

Calculating a security's rate of return

The main goal of every investor is to earn a good rate of return on his investment. Imagine that you bought a share of Apple's equity at the beginning of the year. Back then, Apple shares were trading at \$105, whereas at the end of the year, their price increased to \$160. Taking into account any transaction costs and dividends paid by the company throughout the year, if you sell that share, you will receive \$116, so you've made a profit of \$11.

To decide whether we should be happy with this profit, we should

compare our investment in Apple with other investments. All we have to do is to calculate a percentage rate of the return of the investment. Once we do that, we can compare it to other investments. The rate of return of an investment is given by the following formula:

$$\text{Rate of Return} = \frac{\text{End Price} - \text{Beginning Price}}{\text{Beginning Price}}$$

This is as follows:

$$\frac{\text{End Price} - \text{Beginning Price}}{\text{Beginning Price}} = \frac{\text{End Price}}{\text{Beginning Price}} - 1$$

In our case, this will be \$116 minus \$105, divided by \$105. This gives us a 10.5% rate of return. This computation of the rate of return is called a **simple rate of return**. If we assume that Apple paid a \$2 dividend at the end of the year, the rate of return calculation becomes the following:

$$\frac{(\$116 + \$2) - \$105}{\$105} = 12.4\%$$

We can calculate the logarithmic return of the investment as follows:

$$\ln\left(\frac{\text{End Price}}{\text{Beginning Price}}\right)$$

$$\log\left(\frac{\$116}{\$105}\right) = \log(\$116) - \log(\$105) = 10\%$$

The value of 10% was attained after multiplying the value with 100.

Note that mathematicians writing $\log x$ usually mean $\log_{10} x$, also called $\ln x$. The reason we use log is specified next.

It is important to be consistent with the way that we calculate returns. If we choose to calculate simple returns, we have to do this for all further financial calculations. Similarly, if we decide to calculate log returns, we should use only log returns. There isn't a general rule as to the method we should use, but most econometricians believe that a simple rate of returns is preferable when you have to deal with multiple assets over the same timeframe and a logarithmic rate of returns is preferable when you make calculations about a single asset over a period of time.

We should always remember the timeframe for which we've calculated a rate of return. In our example, we have calculated a yearly rate of

return, because the Apple stock was held for a year and then sold. Typically, investors use daily, monthly, quarterly, or yearly returns. The most popular timeframe is annually. We can easily convert daily, monthly, and quarterly returns to yearly returns using the following basic formula:

$$\text{annual return} = [(\text{daily return} + 1)^{365}] * 100$$

Converts daily, monthly, and quarterly returns to yearly

In the next section, we will calculate the rate of returns of a security using Python.

Calculating the rate of return of a security in Python using simple returns

In the previous section, we learned how to calculate the rate of return of a security using simple rate of returns and logarithmic rate of returns from a theoretical perspective. Let's now move on and see how we can calculate the simple rate of return for a security using Python. In this section, we are going to use the stock prices of Microsoft (MSFT). This dataset was downloaded from Yahoo! Finance for the period between 2000 and 2017. First, we will import the necessary libraries such as numpy, pandas, and matplotlib, as shown in the following code block:

```
In [1]:
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Then, we are going to read the Microsoft stock prices.

Let's read the dataset, as shown in the following code block

```
In [3]:
MSFT = pd.read_csv('MSFT_stock.csv', index_col = 'Date')
```

The following code helps us to see the top five records of the MSFT dataframe:

```
In [4]:
MSFT.head()
```

The output is as follows:

Date	Open	High	Low	Close	Adj Close	Volume
1999-12-31	58.75000	58.8750	58.1250	58.37500	38.771053	12517600
2000-01-03	58.68750	59.3125	56.0000	58.28125	38.708794	53228400
2000-01-04	56.78125	58.5625	56.1250	56.31250	37.401215	54119000
2000-01-05	55.56250	58.1875	54.6875	56.90625	37.795563	64059600
2000-01-06	56.09375	56.9375	54.1875	55.00000	36.529484	54976600

The following code helps us to see the tail, or the last five records, of the MSFT dataframe:

```
In [5]:
MSFT.tail()
```

The output is as follows:

Date	Open	High	Low	Adj	Close	Volume
2017-12-06	81.550003	83.139999	81.430000	82.779999	82.779999	26162100
2017-12-07	82.540001	82.800003	82.000000	82.489998	82.489998	23184500
2017-12-08	83.629997	84.580002	83.330002	84.160004	84.160004	24489100
2017-12-11	84.290001	85.370003	84.120003	85.230003	85.230003	20121700
2017-12-12	85.309998	86.050003	85.080002	85.529999	85.529999	7805031

As discussed in the previous section, the simple rate of returns of the security is as follows:

$$\frac{\text{End Price} - \text{Beginning Price}}{\text{Beginning Price}} = \frac{\text{End Price}}{\text{Beginning Price}} - 1$$

The Python code for calculating the simple rate of returns is shown here:

```
In [6]:
MSFT['simple_return'] = (MSFT['Close'] / MSFT['Close'].shift(1)) - 1
print (MSFT['simple_return'].head(10))
```

```
Out[6]:
Date
1999-12-31    NaN
2000-01-03  -0.001606
2000-01-04  -0.033780
2000-01-05   0.010544
2000-01-06  -0.033498
2000-01-07   0.013068
2000-01-10   0.007291
2000-01-11  -0.025612
2000-01-12  -0.032571
2000-01-13   0.018901
Name: simple_return, dtype: float64 of
```

Here, we are calculating the simple returns on a day-to-day basis using the shift function. The value we provided is 1. The data series shown in the output is as expected: it exhibits the percentage daily change of the closing price. On most days, the number is lower than 1%. Significant movements of a company's stock price are not an everyday occurrence. Note that the first value of the series is **not a number (NaN)**. This makes sense, as there is no lag for our first observation.

Let's plot these transformations using the matplotlib library. Before plotting it, however, we have to convert the index, which is in string format, to datetime, using pandas, as follows:

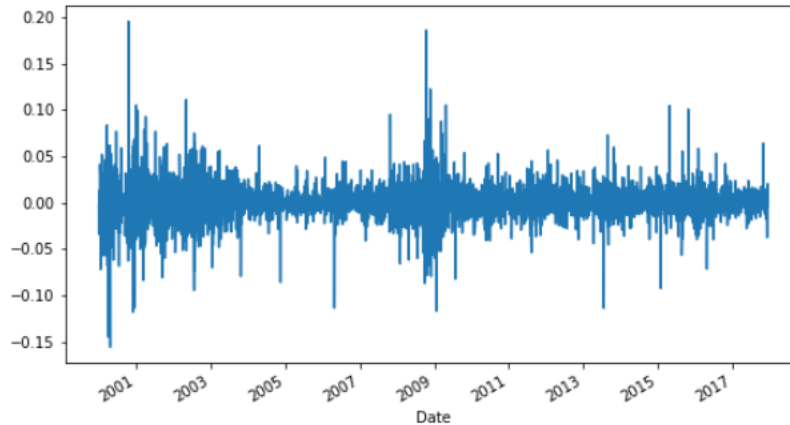
```
In [13]:
MSFT.index=pd.to_datetime(MSFT.index)
```

Then, we will plot the simple returns field calculated for the security, as

shown here:

```
In [14]:  
MSFT['simple_return'].plot(figsize=(9,5))  
plt.show()
```

The output is as follows:



The preceding graph shows the daily simple returns that were observed throughout the defined period. As you can see, the most significant daily observations were mainly negative. In finance, it is common to see negative returns that have a much higher magnitude than positive returns. Usually, positive returns accumulate over time and stock prices increase, but when things do go wrong, stock prices tend to fall very quickly. As you can see from the graph, in 2001, Microsoft stocks experienced a loss of more than 15% in one day.

However, an investor who is interested in buying a stock and holding it in the long run is mainly interested in the average rate of return that the stock will have. For this reason, we calculate the mean return of the MSFT throughout the period under analysis. To do this, we will apply a mean function that calculates the average daily rate of returns. This function is available in pandas, as in the following code:

```
In [16]:
MSFT_average_return=MSFT['simple_return'].mean()
MSFT_average_return

Out[16]:
0.00027
```

The output is a really small number, much smaller than 1%, which makes it very difficult to interpret. We might prefer to find the **average annual rate of return**. The data that we have extracted, however, is not composed of 365 days observations per year. It excludes non-trading days, such as Saturdays, Sundays, and bank holidays. The number of trading days actually comes to between 250 and 252 days. For now, let's use the number 250. The next step is to multiply the average daily return by 250, which will give us a close approximation of the actual average return per year. This value will be easier to

understand than the previous one. The code to do this is shown as follows:

```
In [17]:
MSFT_average_return=MSFT['simple_return'].mean()*250
MSFT_average_return

Out[17]:
0.06820496450977334
```

We will multiply this value by 100 and add a %, as follows:

```
In [19]:
print(str(MSFT_average_return * 100)+ '%')

Out[19]:
6.820496450977334%
```

In this section, we looked at how to estimate the simple market returns of a given stock, how to plot it, and how to turn this into a meaningful value that is easy to understand and interpret.

Calculating a security's rate of return using logarithmic return

Just as a reminder, the logarithm rate of return is given by the following formula:

$$\ln\left(\frac{\textit{End Price}}{\textit{Beginning Price}}\right)$$

Let's see how we can implement this with Python. We will be using the same libraries as we did for simple returns: pandas, matplotlib, and numpy. We will read the same Microsoft stock prices dataset and apply the preceding log formula using Python, as shown here:

```
In [1]:
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

In [2]:
MSFT = pd.read_csv('MSFT_stock.csv', index_col = 'Date')
```

We apply the logarithmic formula previously mentioned using the log function from the numpy package and taking the Close value of a given observation for a certain trading day and dividing it by the Close value of the day before, as shown here:

```
In [3]:
MSFT['log_return'] = np.log(MSFT['Close'] / MSFT['Close'].shift(1))
print (MSFT['log_return'])
```

The output is as follows:

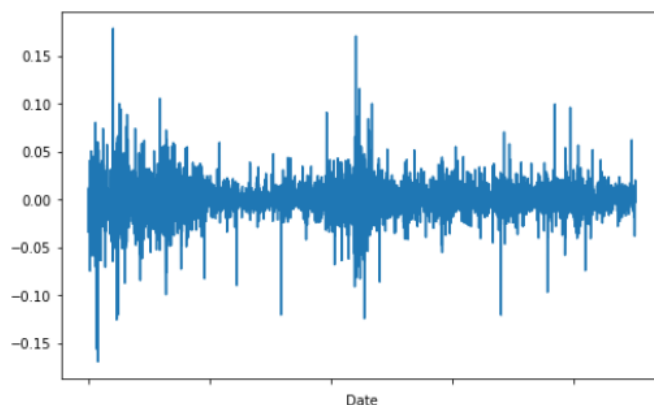
Date	
1999-12-31	NaN
2000-01-03	-0.001607
2000-01-04	-0.034364
2000-01-05	0.010489
2000-01-06	-0.034072
2000-01-07	0.012984
2000-01-10	0.007265
2000-01-11	-0.025946
2000-01-12	-0.033114
2000-01-13	0.018725
2000-01-14	0.040335
2000-01-18	0.026917
2000-01-19	-0.074817
2000-01-20	-0.009390
2000-01-21	-0.021455
2000-01-24	-0.024391
2000-01-25	0.015314
2000-01-26	-0.034006
2000-01-27	-0.006309
2000-01-28	-0.005076
2000-01-31	-0.003824
2000-02-01	0.050431
2000-02-02	-0.020860
2000-02-03	0.027516
2000-02-04	0.027953
2000-02-07	0.000586
2000-02-08	0.030594

From this, we obtain a series and, again, most of the values are less than 1%.

Let's plot the graph:

```
In [4]:
MSFT.index=pd.to_datetime(MSFT.index)
MSFT['log_return'].plot(figsize=(8, 5))
plt.show()
```

The output is as follows:



When we plot the newly obtained data on a graph, we see that the log rate of return ostensibly resembles the simple rate of return. However, when we calculate the average daily and annual log returns, we obtain a percentage that is significantly smaller than the one we obtained for simple returns.

Let's calculate the average mean and the annual log return:

```
In [6]:  
log_return_d = MSFT['log_return'].mean()  
log_return_d
```

```
Out[6]:  
8.45835836828664e-05
```

The following code is used to calculate the annual log returns, considering 250 as the number of working days in a year:

```
In [8]:  
log_return_a = MSFT['log_return'].mean() * 250  
log_return_a
```

```
Out[8]:  
0.0211458959207166
```

The following code is used to convert the annual log to a percentage by multiplying it by 100:

```
In[9]:  
print(str(log_return_annually * 100) + '%')
```

```
Out[9]:  
2.11458959207166%
```

To conclude, it is preferable to use simple returns when we calculate the returns of multiple securities in the same period. On the other hand, log returns are a better choice when we have only one security and we want to calculate the returns over multiple time periods.

Measuring the risk of a security

Investors prefer good rates of return and they don't like risk uncertainty, because this means that there is a chance that they'll lose a large portion of their money. As mentioned previously, risk and return are the two most important dimensions in investment decision making. It is, therefore, easy to understand why we should spend a significant portion of time learning how to measure and forecast security risks. Let's think about how we can define risk for an investor. If you invest \$1,000 of your money in a stock that's trading on the stock exchange

and you know that on average this stock earns 15%, you might want to know what numbers the average is calculated from. Let's consider two different scenarios:

- In one year, the stock earned 14%. In the following year, it earned 16%. In the final two years observed, it earned 13% and 17% respectively.
- In one year, the stock earned 50%. In the following two years, it earned -20%. In the final year observed, it earned 50%.

There is a big difference between these two sets of data. In the first case, you can be certain that your money will earn an amount that is more or less in line with what you expect. Things might be slightly better or slightly worse, but the rate of return will always be between 13% and 17%. In the second set of data, however, although the average return is the same, there is a huge variability from one year to the next. An

investor will be unsure about what might happen next. If they invested their money over the second and third observed years, they would have lost 40% of their initial investment.

This shows that *variability* plays an important role in the world of finance. It is the best measure of risk. The stock market is volatile and is likely to surprise investors, both positively and negatively. Investors, however, don't like surprises and are much more sensitive to the possibility of losing their initial investment. Most people prefer to have a good idea about the rate of return they can expect from a security, or a portfolio of securities, and do their best to reduce the risk that they are exposed to. Our goal is to measure the risk faced by investors and try to reduce it as much as possible.

The most commonly used statistical measures, variance, and standard deviation, can help us a great deal when quantifying the risk associated with the dispersion of the likely outcome.

The variance of a security measures the dispersion of a set of data points around their mean value. It is equal to the following equation:

$$s^2 = \frac{\sum (x - \bar{x})^2}{n - 1}$$

The squared variance is equal to the sum of the squares of the difference between a data point x and the mean divided by the total number of data points minus one. If we take the square root of the variance, we get the standard deviation of this sample of observations:

$$s = \sqrt{s^2}$$

Let's go ahead and calculate the stock's variance and standard deviation using Python in the next section.

Calculating the risk of a security in Python

In this section, we will look at how to calculate a security risk. We will use the Adjusted Close column of MSFT and AAPL and we will use the variance and standard deviation. First, we will import the same libraries: numpy, pandas, and matplotlib:

```
In [1]:
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Then, we will import the dataset using pandas:

```
In [2]:
security_data = pd.read_csv('MSFT_AAPL_stock.csv', index_col='Date')
```

After that, we will examine the behavior of the two stocks over the past 17 years by retrieving data from December 31, 1999:

```
In [3]:
security_data.index = pd.to_datetime(security_data.index)

In [4]:
security_data.head()
```

The output is as follows:

	MSFT	AAPL
Date		
1999-12-31	38.965767	3.303425
2000-01-03	38.903194	3.596616
2000-01-04	37.589046	3.293384
2000-01-05	37.985374	3.341579
2000-01-06	36.712940	3.052405

The standard deviation of a company's returns can also be called the **risk** or **volatility**. A stock whose returns show a large deviation from its mean is said to be more volatile.

Let's take a look at which company's stocks are riskier or more volatile. First, we take the logarithmic returns, because we will examine each company separately in the given timeframe. This approach will tell us more about the behavior of the stock:

```
In [7]:  
security_returns = np.log(security_data / security_data.shift(1))
```

We can use the head function to see the top 20 records, as shown in the following code:

```
In [9]:  
security_returns.head(20)
```

The output is as follows:

	MSFT	AAPL
Date		
1999-12-31	NaN	NaN
2000-01-03	-0.001607	0.085034
2000-01-04	-0.034364	-0.088078
2000-01-05	0.010489	0.014528
2000-01-06	-0.034072	-0.090514
2000-01-07	0.012983	0.046281
2000-01-10	0.007264	-0.017745
2000-01-11	-0.025946	-0.052505
2000-01-12	-0.033114	-0.061847

2000-01-13	0.018725	0.104069
2000-01-14	0.040335	0.037405
2000-01-18	0.026918	0.034254
2000-01-19	-0.074817	0.024942
2000-01-20	-0.009390	0.063071
2000-01-21	-0.021455	-0.019461
2000-01-24	-0.024391	-0.046547
2000-01-25	0.015314	0.054934
2000-01-26	-0.034007	-0.018545
2000-01-27	-0.006309	-0.001703
2000-01-28	-0.005076	-0.079191

We will store this data in a variable called **security_returns**. This newly created variable has two columns, each of which contains the log returns of MSFT and AAPL, respectively. This allows us to obtain the mean and the standard deviation of the two stocks for the dataframe.

Then, we apply a mean function from NumPy, as follows:

```
In [10]:
security_returns['MSFT'].mean()

Out[10]:
0.00011508712580239439
```

From this function, we obtain a small number that equals the daily average return. Let's analyze this value by multiplying it by the number of trading days in a year, 250, as follows:

```
In [11]:
security_returns['MSFT'].mean()*250

Out[11]:
0.028771781450598596
```

The output obtained is a value just under 3%, which is the annual rate of return. The same Pythonic logic must be applied to calculate the volatility of the company's stock. The method that we will be using is called **standard deviation**:

```
In [12]:
security_returns['MSFT'].std()

Out[12]:
0.019656948849403607
```

After calculating the standard deviation, we will multiply the output by the square root of 250, which represents the trading days. This square root of 250 is taken because the standard deviation is the square root of the variance:

```
In [18]:
security_returns['MSFT'].std()*250**0.5

Out[18]:
0.31080365106770774
```

We then repeat the same procedure for AAPL; we get a lower mean but a higher volatility percentage:

```
In [14]:
security_returns['AAPL'].mean()

Out[14]:
0.000864342049190355
```

The following code help us to calculate for the annual rate of return:

```
In [15]:
security_returns['AAPL'].mean()*250

Out[15]:
0.21608551229758874

In [16]:
security_returns['AAPL'].std()

Out[16]:
0.027761934312069386

In [19]:
security_returns['AAPL'].std()*250**0.5

Out[19]:
0.4389547233905951
```

It will be easier for us to interpret the results if we quoted the two means in the two standard deviations next to each other. To do this, we can print the equations of the two annual means and standard deviations, as shown in the following code:

```
In [20]:
print(security_returns['MSFT'].mean()*250)
print(security_returns['AAPL'].mean()*250)
```

The output for this is as follows:

```
Out[20]:
0.028771781450598596
0.21608551229758874
```

The following code helps us to see the top five records:

```
In [22]:
security_returns.head(5)
```

The output is as follows:

	MSFT	AAPL
Date		
1999-12-31	NaN	NaN
2000-01-03	-0.001607	0.085034
2000-01-04	-0.034364	-0.088078
2000-01-05	0.010489	0.014528
2000-01-06	-0.034072	-0.090514

```
In [23]:
security_returns[['MSFT','AAPL']].mean()*250

Out[23]:
MSFT    0.028772
AAPL    0.216086
dtype: float64

In [24]:
security_returns[['MSFT','AAPL']].mean()*250**0.5

Out[24]:
MSFT    0.001820
AAPL    0.013666
dtype: float64
```

Stocks with a higher expected return often attract more buyers. The Microsoft rate of return is slightly higher, but this comes at the expense of a higher volatility.

Portfolio diversification

In this section, we will talk about one of the most important concepts in finance: the relationship between financial securities. It is reasonable to expect the prices of shares in a stock exchange to be influenced by the same factors. The most obvious example of this is the development of the economy. In general, favorable macro-economic conditions facilitate the business of all companies. When people have jobs and money in their pockets, they will spend more. Companies benefit from this as their revenues increase.

In a tough economy, consumers reduce their spending and buy mainly products of primary importance. Because stock prices are determined

by profits, whenever the economy is doing well, stock prices are higher. Investors will pay a high price for profitable companies. In times of recession, companies profits are lower and share prices fall significantly. However, different industries are influenced in different ways. Let's consider an example; imagine that we have stocks in two industries: one in the automobile sector and one in the food chain sector. Which do you think will suffer more in an economic crisis? The answer is the stock held in the automobile sector. People can't stop buying food and groceries, but they can easily postpone buying a new car and continue driving an old vehicle. Therefore, the state of the economy impacts different industries in different ways. An investor should always focus on building a portfolio of stocks that involves different sectors.

Let's consider another example to further understand stock portfolios:

let's suppose that we own one stock of a tech company, such as Facebook. We then want to buy some shares of a second company. We can either choose LinkedIn or Walmart. For the purposes of this example, let's assume that we expect the same rate of return from both shares. Which one would you choose? The right answer is Walmart, because Walmart operates in a different industry and this gives you protection as an investor. If things don't go well in the internet domain, spending decreases and competition becomes stronger. Technology changes rapidly and any sector can face problems. If this is the case, we would still have the Walmart share, which would not suffer in the same way. The same concept is valid for the retail sector: if Walmart isn't performing as well as expected, we'd still have our Facebook share, which operates in a different industry.

By buying the shares of two companies operating in the same industry, our portfolio will be exposed to an excessive risk for the same level of

domain, spending decreases and competition becomes stronger. Technology changes rapidly and any sector can face problems. If this is the case, we would still have the Walmart share, which would not suffer in the same way. The same concept is valid for the retail sector: if Walmart isn't performing as well as expected, we'd still have our Facebook share, which operates in a different industry. By buying the shares of two companies operating in the same industry, our portfolio will be exposed to an excessive risk for the same level of expected return. There is clearly a relationship between the prices of different companies. It is very important to understand what causes this relationship and how to use this measurement to build optimal investment portfolios.

Covariance and correlation

Now that we know that it is reasonable to expect a relationship between the returns of different stocks, we have to learn how to quantify this relationship. Let's take a look at an example about the factors that determine the price of a house. One of the main factors is the size of the house: the larger it is, the more expensive it is. There is clearly a relationship between these two factors. Statisticians use the term **correlation** to measure this relationship. Usually, this correlation output of the calculation lies in the interval from -1 to +1.

To understand this concept better, let's take a look at the formula that allows us to calculate the covariance between two variables:

$$\sigma_{xy} = \frac{(x - \bar{x}) * (y - \bar{y})}{n - 1}$$

Here, $\bar{x}$ is the mean of factor x (size) and $\bar{y}$ is the mean of factor y (price). n is the number of records, which contains the combination of x and y . The correlation coefficient measures the relationship between the two variables.

The output of the covariance specifies the following information:

- If the covariance is more than zero, the two variables move in the same direction.
- If the covariance is less than zero, the two variables move in the opposite direction.
- If the covariance is equal to zero, the two variables are independent.

Let's now discuss correlation; correlation is the concept, and correlation coefficient is the measure of correlation. It has values that range from -1 to 1, where -1 or 1 indicate a perfect linear association and 0 indicates no

linear relationship. The relationship between the two variables, or the two factors, becomes easy to interpret. Let's consider the previous example again. While it is true that the price of a house is directly proportionate to the size of the house, in reality, several variables have an impact, such as their location and their year of construction. There is a similar story when we compare company shares. There are several variables that determine share prices, such as industry growth, revenue growth, profitability, regulatory environment, and growth perspective. The more similar the context in which two companies operate, the more correlation there will be between their share prices, as they will be influenced by the same, or similar, factors.

Usually, correlation is given by the following formula:

$$\rho_{xy} = \frac{(x - \bar{x}) * (y - \bar{y})}{\sigma_x \sigma_y}$$

$$\rho_{xy} = \frac{\sigma_{xy}}{\sigma_x \sigma_y}$$

There are three different types of correlation output, as follows:

- **Perfect correlation:** A correlation of a value of one, also known as perfect positive correlation, means that the entire variability of the second variable is explained by the first variable. For example, house prices are directly proportionate to house size if we are considering house size as the only parameter affecting house price.
- **No correlation:** A correlation of a value of zero between two variables means they are independent from each other. For example, we will expect a correlation of zero between the price of coffee in Brazil and the price of houses in the USA. These two variables have nothing in common.
- **Negative correlation:** We can either have a perfect negative correlation of a value of -1, or, as is much likelier, an imperfect negative correlation of a value between -1 and 0. For example, there

might be a negative correlation between the profits of a company producing ice cream and a company selling umbrellas. More ice cream is sold when the weather is good, while more umbrellas are sold when it's rainy. This is an example of a situation where the prices of two companies are influenced by the same variable, but the variable impacts their businesses in a different way.

Using the preceding formula, we can calculate a correlation coefficient manually. In the next section, we will learn how to calculate the covariance and the correlation between the prices of AAPL and MSFT quickly and easily using Python.

Calculating the covariance and correlation

Let's take a look at how we can apply this knowledge in Python. Now that you know what covariance is, you should be able to grasp the concept of a covariance matrix more easily. A covariance matrix is a representation of the way that two or more variables relate to each other, as shown in the following formula:

$$\text{Covariance Matrix : } \Sigma = \begin{bmatrix} \sigma_1^2 & \sigma_{12} & \cdots & \sigma_{1I} \\ \sigma_{21} & \sigma_2^2 & \cdots & \sigma_{2I} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{I1} & \sigma_{I2} & \cdots & \sigma_I^2 \end{bmatrix}$$

The covariance between a variable and itself is the variance of that variable. Along the main diagonal, we have the variances of different variables. The rest of the table should be filled with the covariances between them. In our case, the variables are the prices of two stocks. We will therefore expect a two-by-two covariance matrix with the variances of each stock along the main diagonal, and the covariance between the two stocks displayed in the other two cells. In Python, there is no need to do mathematical calculations manually. The `var()` method calculates the variance of the object for us. We will use this function to calculate the variances of the stock prices of Microsoft and Apple. The code is as follows:

```
In [8]:
MSFT = security_returns['MSFT'].var()
MSFT

Out[8]:
0.00038639563806806983
```

The following code helps us to calculate the variance of the Apple stock:

```
In [9]:
AAPL = security_returns['AAPL'].var()
AAPL

Out[9]:
0.0007707249967476555
```

After that, we make these values annual, as shown in the following code block:

```
In [10]:
MSFT = security_returns['MSFT'].var() * 250
MSFT
```

```
Out[10]:
0.09659890951701745
```

```
In [11]:
AAPL = security_returns['AAPL'].var()*250
AAPL
```

```
Out[11]:
0.19268124918691387
```

In order to compute the pairwise covariance of the column of the stock price, we will use the `cov()` function, which is available in the pandas library; the code for this is as follows:

```
In [12]:
cov_matrix = security_returns.cov()
cov_matrix
```

The output is as follows:

MSFT	AAPL	
MSFT	0.000386	0.000218
AAPL	0.000218	0.000771

Next, we analyze the result to obtain the annual covariance matrix, as shown here:

```
In [13]:
cov_matrix = security_returns.cov()*250
cov_matrix
```

The output is as follows:

	MSFT	AAPL
MSFT	0.096599	0.054592
AAPL	0.054592	0.192681

Let's examine this matrix cell by cell, starting from the top-left corner. The top-left corner has the same values as that of the MSFT annualized variance value, which was computed by the `var()` function mentioned previously. The `cov()` method is useful because it allows us to obtain the other numbers easily.

Let's now calculate the correlation with the help of the `corr()` method,

which is available in pandas, as shown here:

```
In[14]:  
corr_matrix = security_returns.corr()  
corr_matrix
```

The output is as follows:

```
Out[14]:
```

	MSFT	AAPL
MSFT	1.000000	0.400153
AAPL	0.400153	1.000000

The `corr()` computes the correlation of columns pairwise. From the preceding output, we can see that the first diagonal element is 1, because we divide the variances of MSFT and AAPL by the same values. It makes sense that the movement of the stock is perfectly correlated with itself. The other two cells in the correlation table contain the same number. This tells us that the stock returns of the two companies are weakly correlated. Be very careful here: this is not the correlation between the price of the two equities. This often creates confusion. When carrying out financial analysis, the correlation between prices and the correlation between returns may show different values. They usually have different implications and it is important to respect this distinction. The correlation between the returns, which is the same as the correlation between the rate of returns, reflects the dependence between prices at different times and focuses on the returns of your portfolio rather than on the stock price levels.

Summary

In this chapter, we have looked at a wide range of concepts regarding measuring investment risks. We started by exploring the various sources of a financial dataset and looked at where we can download datasets from. We then learned about both the simple rate of returns for a security and the logarithmic rate of returns, and saw how we can compute these values using Python on the dataset of the adjusted column of the stocks of Microsoft and Apple. After that, we looked at how to calculate the risk of a security both theoretically and practically, using Python. Finally, we explored the concept of portfolio diversification and discussed the concepts of covariance and correlation to find the variance between the stock prices of the two companies, Microsoft and Apple.

In the next chapter, we will be discussing portfolio allocation and Markowitz portfolio optimization, which will help us to allocate and optimize our portfolios efficiently.